

Отчёт: Monitoring + Alerting + Telegram для MEGA CODER

1. Цель работы

Цель доработки — превратить существующий DevOps-стенд в production-like observability решение: метрики собираются Prometheus, правила формируют alerts, Alertmanager группирует события, а отдельный webhook-сервис отправляет уведомления в Telegram. Дополнительно добавлен reporter, который формирует Markdown-отчёт по Kubernetes, приложению, метрикам и логам.

Важно: новые компоненты сделаны изолированно и по умолчанию выключены через `alerting.enabled=false` в `helm/mega-coder/values.yaml`, чтобы не сломать уже рабочий стенд перед сдачей.

2. Архитектура решения

Потоки событий:

1. `PrometheusRule` описывает условия проблем: CrashLoopBackOff, restart spike, mismatch реплик, TargetDown и другие.
2. Prometheus отправляет сработавшие alerts в Alertmanager.
3. Alertmanager группирует alerts по `alertname`, `namespace`, `severity`.
4. Alertmanager вызывает webhook `alert-bot` по адресу `/webhook/alertmanager`.
5. `alert-bot` форматирует сообщение и отправляет его в Telegram.
6. GitLab webhooks могут идти в тот же сервис по адресу `/webhooks/gitlab`.
7. Reporter запускается как Kubernetes `CronJob` или вручную и формирует Markdown-отчёт.

Компоненты в репозитории:

```
| Компонент | Путь | Назначение |
|---|---|---|
| Alert bot | `services/alert-bot` | FastAPI webhook receiver для Alertmanager/Grafana/GitLab |
| Reporter | `services/reporter` | Markdown-отчёт по Kubernetes/Prometheus/Loki |
| Helm templates | `helm/mega-coder/templates/*alert*`, `cronjob-reporter.yaml` | Опциональный деплой alerting-компонентов |
| Prometheus rules | `monitoring/prometheus/rules/mega-coder-alerts.yaml` | Standalone rules для демонстрации и ручного apply |
| Alertmanager config | `monitoring/alertmanager/alertmanager.yaml` | Пример receiver/route до Telegram bridge |
| Examples | `examples/` | Payloads, secrets example, values overlay |
```

3. Почему выбран Prometheus + Alertmanager + Telegram

Prometheus уже является частью monitoring stack, поэтому логично добавлять alerts именно на его стороне. Alertmanager выбран как стандартный компонент для группировки, подавления дублей, отправки resolved-событий и маршрутизации. Telegram выбран как простой канал incident notification: уведомления видны на телефоне, легко показать на защите, не требуется отдельная корпоративная почта или платный сервис.

4. Как создан Telegram-бот

Бот создаётся через `@BotFather`, после чего token сохраняется только в Kubernetes Secret. Токен из личного чата пользователя не сохранён в репозитории. Подробная инструкция есть в `BOT\_SETUP.md`.

5. Как настроены секреты

Секреты не хранятся в git. Используются переменные:

```
| Env | Для чего |
|---|---|
| `TELEGRAM_BOT_TOKEN` | token Telegram bot |
| `TELEGRAM_CHAT_ID` | chat/group id |
| `TELEGRAM_PARSE_MODE` | `HTML` по умолчанию |
| `ALERTMANAGER_WEBHOOK_SECRET` | необязательный shared secret для Alertmanager/Grafana/report webhook |
| `GITLAB_WEBHOOK_SECRET` | secret для GitLab webhook |
| `REPORT_NAMESPACE` | namespace для reporter |
| `PROMETHEUS_URL` | URL Prometheus внутри кластера |
| `LOKI_URL` | URL Loki внутри кластера |
```

Для безопасного включения используется secret `mega-coder-alerting-secret`, пример структуры лежит в `examples/secrets-example.yaml`.

6. Как настроен Alertmanager

Файл `monitoring/alertmanager/alertmanager.yaml` задаёт:

```
- `group_by: ["alertname", "namespace", "severity"]`;
- `send_resolved: true`;
```

- webhook receiver `telegram-alert-bot`;
- внутренний URL `http://mega-mega-coder-alert-bot.mega-coder.svc.cluster.local:8088/webhook/alertmanager`.

На текущем стенде config можно применить через values kube-prometheus-stack или использовать как evidence на защите.

7. Как настроены правила Prometheus

Реализованы alerts:

```
| Alert | Severity | Источник |
|---|---|---|
| `PodCrashLooping` | critical | kube-state-metrics |
| `PodRestartTooOften` | warning | kube-state-metrics |
| `DeploymentReplicasMismatch` | critical | kube-state-metrics |
| `PodNotReady` | warning | kube-state-metrics |
| `HighCPUUsage` | warning | cAdvisor/container metrics |
| `HighMemoryUsage` | warning | cAdvisor/container metrics |
| `TargetDown` | critical | Prometheus `up` |
| `TooMany5xx` | warning | ingress-nginx metrics, если установлен ingress |
| `LokiErrorSpike` | warning | Loki ruler example |
| `GitLabPipelineFailed` | event notification | GitLab webhook, не Prometheus metric |
```

Каждое правило содержит `summary`, `description`, `severity`, `cluster`, `env`, `app`, `runbook\_url`.

8. Как работает webhook receiver

`services/alert-bot/app/main.py` содержит endpoints:

- `GET /health` — healthcheck.
- `POST /webhook/alertmanager` — основной production path.
- `POST /webhook/grafana` — fallback для Grafana Alerting.
- `POST /webhook/gitlab` — GitLab push/pipeline/MR/tag/release events.
- `POST /webhook/report` — отправка краткого отчёта reporter через тот же Telegram bot.

Сервис экранирует HTML-символы, поддерживает `critical/warning/info`, умеет показывать `firing/resolved`, группировку, Kubernetes labels и runbook URL.

На домашнем k3s-стенде для `alert-bot` включён точечный `hostNetwork` через `examples/values-alerting-enable.yaml`: с хоста Telegram API доступен, а прямой egress из pod-сети до Telegram блокировался провайдером/локальным проху-маршрутом. В базовом `values.yaml` этот режим выключен, поэтому обычный production-like deploy остаётся безопасным и предсказуемым.

9. Как реализованы отчёты

`services/reporter/app/report.py` запускается как CronJob или вручную. Он читает Kubernetes API через service account, собирает:

- список pod'ов;
- restart count;
- unavailable deployments;
- Warning/Error events;
- CPU/memory/availability через Prometheus;
- ERROR-сэмпл через Loki;
- итоговую сводку приложения.

По умолчанию reporter не спамит Telegram: `sendReportToTelegram=false`, расписание редкое `0 \*/6 \* \* \*`. Для защиты лучше запускать вручную.

10. Как подключены события GitLab

GitLab webhook настраивается на:

- push events;
- pipeline events;
- merge request events;
- tag push/release events, если используются.

Событие “new user registered” в обычном project webhook недоступно штатно, поэтому оно честно не заявляется как реализованное. Вместо него реализованы реальные и демонстрируемые события проекта: pipeline failed/success, push, MR.

11. Сценарии демонстрации

1. Открыть `README.md`, показать архитектуру.
2. Открыть `services/alert-bot/app/main.py`, показать endpoints.
3. Открыть `monitoring/prometheus/rules/mega-coder-alerts.yaml`, показать alert rules.
4. Открыть `helm/mega-coder/values.yaml`, показать `alerting.enabled=false` как безопасный feature flag.
5. Создать secret по `BOT\_SETUP.md`.
6. Включить alerting overlay `examples/values-alerting-enable.yaml`.
7. Отправить тестовый payload из `examples/alertmanager-firing.json`.
8. Показать Telegram firing/resolved.
9. Показать reporter job и Markdown-отчёт.

12. Примеры уведомлений

Пример firing:

```
```text
```

● Alertmanager: СРАБАТЫВАЕТ  
алертов в группе: 1

● СРАБАТЫВАЕТ critical / критично — PodCrashLooping  
namespace: mega-coder  
объект: mega-mega-coder-api-demo  
кратко: Pod находится в CrashLoopBackOff  
описание: Тестовый pod постоянно перезапускается: это демонстрационный alert для проверки Telegram-интеграции.

Пример GitLab:

```
```text
```

● GitLab pipeline: ОШИБКА
проект: MEGA/devops-mega-coder
ветка/tag: main
commit: 76475c5
ссылка: <https://gitlab.ru/MEGA/devops-mega-coder/-/pipelines/23>

13. Примеры отчётов

Шаблон отчёта лежит в `examples/report-example.md`. Реальный live-отчёт сохранён в `docs/evidence/reporter-live.md`.

Новый отчёт можно получить из CronJob logs:

```
```bash
```

```
sudo k3s kubectl create job -n mega-coder --from=cronjob/mega-mega-coder-reporter reporter-manual-demo
sudo k3s kubectl logs -n mega-coder job/reporter-manual-demo
```

## ## 14. Возможные проблемы и устранение

| Проблема | Что проверить |

|---|---|

Telegram не приходит	secret `TELEGRAM\_BOT\_TOKEN`, `TELEGRAM\_CHAT\_ID`, bot started by user
Alertmanager не вызывает bot	service `mega-mega-coder-alert-bot`, receiver URL, network policy
Из pod нет доступа к Telegram, а с хоста есть	включить `alerting.alertBot.hostNetwork=true` только для домашнего стенда
PrometheusRule не виден	label `release=monitoring`, CRD `PrometheusRule`, kube-prometheus selector
GitLab webhook 401	`X-Gitlab-Token` должен совпадать с `GITLAB\_WEBHOOK\_SECRET`
Reporter пустой	service account RBAC, namespace `REPORT\_NAMESPACE`, Prometheus/Loki URLs

## ## 15. Что можно улучшить дальше

- Добавить real SLO alerts по latency/error-rate, если приложение начнёт экспортировать `/metrics`.
- Включить Loki ruler в Helm values и применять `LokiErrorSpike` автоматически.
- Добавить Alertmanager inhibit rules для подавления шумных cascaded alerts.
- Отправлять отчёты по расписанию только в отдельный Telegram topic.

## ## 16. Вывод

Проект получил расширяемую observability-архитектуру: Prometheus rules, Alertmanager route, Telegram webhook bridge, GitLab event bridge, Markdown reporter, Helm templates и документацию. Решение безопасно для текущего стенда, потому что выключено по умолчанию и

включается отдельным values overlay.